

## total-ftp

Based off [Factoring “short-sleeve” RSA keys with polynomials](#) by Keegan Ryan (Trail of Bits)

### article appendix: $n_2$

We know that  $n_2$  is generated with the CompleteFTP RNG as follows:

```
public void genRandomBits(int bits) {  
    // Calculate the number of limbs  
    int numLimbs = bits / 32;  
    // Allocate space for the RNG output  
    byte[] array = new byte[numLimbs];  
    // Call the system RNG  
    rngProvider.GetNonZeroBytes(array);  
    // Copy to the limbs of the big number  
    Array.Copy(array, 0, bignumLimbs, 0, numLimbs);  
    // Set the top bit to ensure proper bit length  
    bignumLimbs[numLimbs - 1] |= 0x80000000;  
    // Store the length  
    dataLength = numLimbs;  
}
```

As discussed in the (amazing) original article, the vulnerability lies in the fact that there is “a mismatch between the size of the limbs and the size of the RNG output”.

1. If you compute  $f_{n_2}(x)$  using  $B = 2^{32}$ , some of the coefficients are large. Why is that? Is it true that all of the coefficients of  $f_p(x)$  and  $f_q(x)$  are small?

Consider the coefficients of  $f_p(x)$  and  $f_q(x)$ , i.e. the 32-bit limbs of  $p$  and  $q$ . From the definition of `genRandomBits()`, we know that each limb is actually a zero-extended 8-bit value, hence  $x \in [1, 255] \cap \mathbb{Z}$  **except the final limb**. The final limb (the most significant, assuming little-endian) has its top bit set. So the result of `genRandomBits(32 * 4)` would look like `[0xab, 0xcd, 0xef, 0x80000012]`.

Therefore it is not true that all of the coefficients of  $f_p(x)$  and  $f_q(x)$  are small: we know that the last coefficient of both will be the full 32 bits. Because this is the

Let  $N_i$  be the  $i$ -th 32-bit limb of  $n_2$ . By polynomial multiplication:

$$N_i = \sum_{j=0}^i p_j q_{i-j}$$

This implies that  $\forall i \geq 31$ ,  $N_i$  must contain multiples of the final limb of  $p$  and  $q$ . So we can expect  $N_i$  to be more than 32-bit for  $i \geq 31$ . In practice due to carrying the values may be less than 32-bit, as we can observe from the limbs as shown below:



## article appendix: $n_1$

note 2026-07-23: this is incomplete and not entirely sure if this is even correct

We can utilise a similar method to solve for  $n_1$ .  $p$  and  $q$  are generated as follows:

```
import random
import Crypto.Util.number
def gen_n1_prime():
    count = 1024 // 128
    w = 128
    b = 32
    p = 0
    for i in range(count):
        pi = random.randrange(2**b)
        p += 2**(w*i) * pi
    p <= 96
    while not Crypto.Util.number.isPrime(p):
        p += 1
    return p
```

We can observe a few differences to the CompleteFTP prime RNG:

- Limb size is 128-bit vs 32-bit
- We generate 32-bits instead of 8-bits
- We do a left shift by 96 bits at the end
  - this is interesting: does this make our primes 1120-bit instead?
  - **no**: `range(count)` goes from  $[0, 7]$ , so  $p_7$  is at bits  $[128 \times 7, 128 \times 7 + 32) = [869, 928)$ . Hence the `p <= 96` causes the top end of  $p_7$  to be at bits  $[992, 1024)$ , making this a 1024-bit prime

Let's revisit questions 1-3 to try and attack this prime.

1. If you compute  $f_{n_1}(x)$  using  $B = 2^{128}$ , some of the coefficients are large. Why is that? Is it true that all of the coefficients of  $f_p(x)$  and  $f_q(x)$  are small?

We have these 128-bit limbs from  $n_1$ :

```
x=89ac77db00000000000000000000000036a77 bit_length=128
x=f0f7d3bfdce03d6000000000000000026b bit_length=124
x=c5e55654e9b14cba000000000000000040f bit_length=128
x=c495b3690a69b66c0000000000000000d9 bit_length=128
x=a82a59342aadff57000000000000000295 bit_length=128
x=1291f9aaa7e9a7d60000000000000000337 bit_length=125
x=d762d68bcbec8cc3a0000000000000000d3 bit_length=128
x=455c59eec3a0654500000000000000003f8 bit_length=127
x=de525d2b1011ceae0000000000000000424 bit_length=128
x=6ee98c677dfdf19000000000000000002 bit_length=123
x=e8e3c0efdd8054ba000000000000000003 bit_length=128
x=2f743377005a840d000000000000000001 bit_length=126
x=1036d671407a066000000000000000002 bit_length=121
x=2c16eeaeab96ddc8000000000000000002 bit_length=126
x=4d2ee8284c7a03c0000000000000000001 bit_length=127
x=c889f7ef523b08e4000000000000000001 bit_length=128
```

We might expect the 32-bit limbs of  $p$  and  $q$  to produce 64-bit limbs of  $n_1$ , but because  $p$  and  $q$  have been left shifted, the 32-bit have ended up in the MSBs of each limb. So in effect we get two 128-bit numbers being multiplied, with carries as seen in the LSBs of the  $n_1$  limbs.

2. Is there a bit shift  $p \ll i$  such that  $f_{2^i p}(x)$  has small coefficients? This is the key trick needed to turn arbitrary short-sleeve values into polynomials with small coefficients.

Let's generate a prime ourselves and take a look at its limbs:

```

n1_prime.bit_length()=1024
x=b8877081000000000000000000000000a7 bit_length=128
x=24a8d64d000000000000000000000000 bit_length=126
x=7b9a1784000000000000000000000000 bit_length=127
x=ef4746e6000000000000000000000000 bit_length=128
x=36260048000000000000000000000000 bit_length=126
x=2e86aebb000000000000000000000000 bit_length=126
x=251d7913000000000000000000000000 bit_length=126
x=dab01506000000000000000000000000 bit_length=128

```

As expected, the bottom 96 bits are empty, with exception of the least-significant limb having its low bits set to make it a prime.

But what if we take 32-bit limbs?

```

x=000000a7 bit_length=8
x=00000000 bit_length=0
x=00000000 bit_length=0
x=b8877081 bit_length=32
x=00000000 bit_length=0
x=00000000 bit_length=0
x=00000000 bit_length=0
x=24a8d64d bit_length=30
x=00000000 bit_length=0
x=00000000 bit_length=0
x=00000000 bit_length=0
x=7b9a1784 bit_length=31
x=00000000 bit_length=0
x=00000000 bit_length=0
x=00000000 bit_length=0
x=ef4746e6 bit_length=32
x=00000000 bit_length=0
x=00000000 bit_length=0
x=00000000 bit_length=0
x=36260048 bit_length=30
x=00000000 bit_length=0
x=00000000 bit_length=0
x=00000000 bit_length=0
x=2e86aebb bit_length=30
x=00000000 bit_length=0
x=00000000 bit_length=0
x=00000000 bit_length=0
x=251d7913 bit_length=30
x=00000000 bit_length=0
x=00000000 bit_length=0
x=00000000 bit_length=0
x=dab01506 bit_length=32

```

Perfect! Every fourth limb is 32-bit, and we have an 8-bit (in this case) least-significant limb.

## Responses to appendix

**4. If this polynomial factorization technique worked for every  $p$  and  $q$ , then RSA would be broken. Why is the short-sleeve property important, and why doesn't this factorization method work in general? What are the limits?**

Note that integer polynomial factorization (that is, factorization over  $\mathbb{Z}$ ) depends on two factors (no pun intended): coefficient size and degree of polynomial.

For short-sleeve RSA moduli, we exploited the fact that the largely uncovered limbs gave rise to small coefficients, using our suitable value of limb size. Hence, we can construct a polynomial representation of the integer with relatively small coefficients and degree so that polynomial factorization is “fast”. Indeed, the short-sleeve property is important.

For uncorrupted RSA moduli, we do not have such small coefficient property. As an *a priori* bound for estimating the complexity of factorization algorithms, we can consider the Landau-Mignotte bound, plagiarized from [Landau-Mignotte bound](#) by Wikipedia:

Let  $f(x), h(x) \in \mathbb{Z}[x]$ , where  $h(x)$  divides  $f(x)$ . Denote the sum of the absolute values of the coefficients in  $h(x)$  and  $f(x)$  as  $\|h\|_1$  and  $\|f\|_1$ , respectively. We then have

$$\|h\|_1 \leq 2^n \|f\|_1$$

where  $n = \deg f(x)$ .

Thus, if  $\|f\|_1$  is large, then we have a large bound for  $\|h\|_1$ , which means we have a large coefficient space to traverse through. As far as I know, most algorithms that factorize univariate integer polynomials require a specified bound for the coefficient of the factors (link: [Factoring univariate polynomials over the integers](#)), so the complexity of factorization is dependent on this bound.

We use the Landau-Mignotte bound to informally justify why uncorrupted RSA moduli always produce a large bound, hence a “slow” factorization, with two cases below:

1. We choose a large limb for a low degree polynomial. Then the coefficients of  $f_n(x)$  are very large, so  $\|f\|_1$  is large, which yields a large coefficient space for its factors.
2. We choose a small limb for small coefficients. Then the degree of  $f_n(x)$  is large, so the  $2^n$  factor in the inequality dominates, which yields a large coefficient space for its factors.

In both cases, we have a large coefficient space for the factors of  $f_n(x)$ , hence why factorization is infeasible for uncorrupted RSA.

**5. The short-sleeve property allows us to construct the product  $f_{2^i p}(x) * f_{2^j q}(x)$ , but unless  $f_{2^i p}(x)$  and  $f_{2^j q}(x)$  are irreducible, factorization may split this into more than two terms. Prove that there is always an efficient way to recover  $p$  and  $q$  from the polynomial factorization.**

Note that `short-sleeve_rsa2.py` gives code on how to do this. We describe the process mathematically below:

Suppose  $f_{2^i p}(x) f_{2^j q}(x)$  factorizes into  $g_1(x) \cdots g_k(x)$  for integer  $k \geq 2$ . Note that for the chosen base  $B = 2^{32}$ , we have

$$f_{2^i p}(B) f_{2^j q}(B) = g_1(B) \cdots g_k(B) = 2^{i+j} pq$$

Thus, exactly two of the factors  $g_i(B)$  contains  $p$  or  $q$ , while other factors are powers of 2.

Therefore, we can traverse through all the factors after factorization and check if the factor is of the form  $2^k$  (a power of 2) or  $2^k p$  for prime  $p$ , which is efficient. Then we can extract  $p$  and  $q$  easily.

□